

Computer Architecture (CSE 511/ ECE 511)

Assignment 1

1. [CO1, CO2 & CO3, CO4] Create a configuration script to simulate the system given in the figure below using two different CPU models: **RiscvTimingSimpleCPU()** and **RiscvO3CPU()**. Run benchmark application: “qsort_large from MiBench benchmark suite [1]” in system emulation mode. Unless otherwise specified, the configuration of caches is as given; L1 Data Cache (size = 16 kB, Associativity = 2, latencies = 2); L1 Instruction Cache (size = 16 kB, Associativity = 2, latencies = 2); L2 Cache (size = 512 kB, Associativity = 4, latencies = 10); L3 Cache (size = 1 MB, Associativity = 8, Latencies = 20).

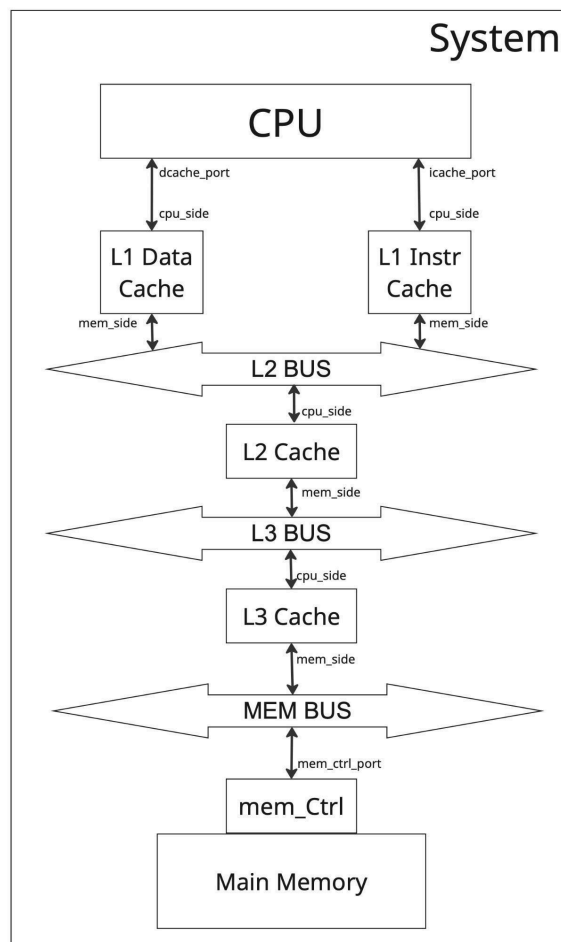


Figure 1: A system architecture with a processor, three cache levels and main memory hierarchies are connected by buses.

Note:

- For this assignment, since you are using RISC-V ISA, you need to build gem5 in RISC-V ISA

using “python3 `which scons` build/RISCV/gem5.opt -j\$(nproc)”

- For running gem5, “ **build/RISCV/gem5.opt configs/tutorial/two_level.py**”. The highlighted area may be different from your config file. This script will serve as a reference and starting point for performing the simulations in this assignment.
- The binary must be compiled using the targeted RISC-V architecture.
- You do not need to give parameters for **RiscvO3CPU()**, use only the gem5 default, if you wish you can give different parameters or branch predictors.

2. **[CO1, CO2 &CO3, CO4]** Perform experimental simulations by changing L2 and L3 cache configurations of the above system with the “**qsort_large**” benchmark to achieve a similar plot as shown in figure (2) below. For your convenience, we have provided a few sets of configurations in Table 1. You can extend the set of configurations to have a better understanding of the system performance by changing the parameters. (Keep the L2 constant while sweeping for the L3 cache and similarly keep the L3 cache constant while sweeping for the L2 cache).

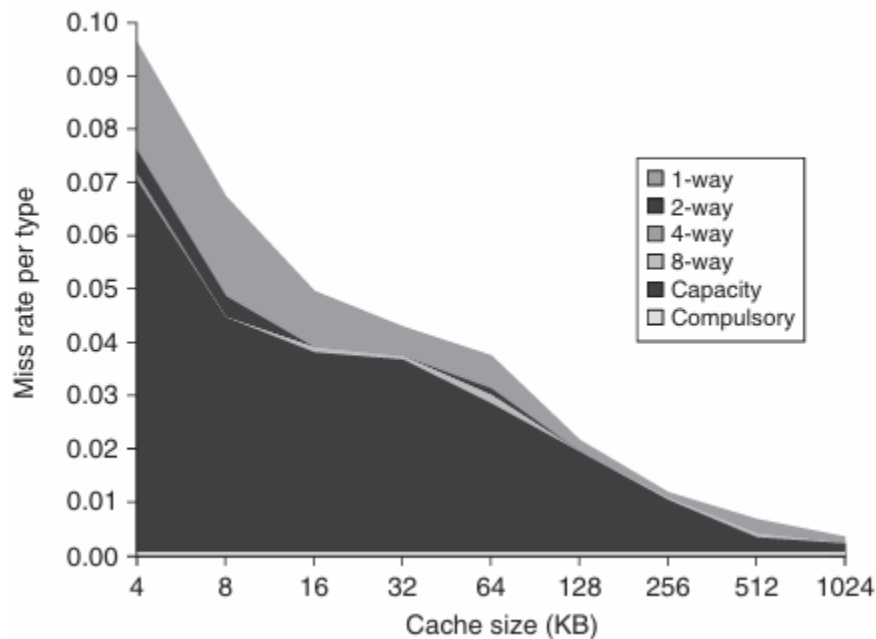


Figure 2: Variation in Cache Miss Rate with different cache sizes and associativity [2].

L2 Configurations (size_associativity)	L2 Configurations (size_associativity)
64 KB_2	1024 KB_8
64 KB_4	1024 KB_2

256 KB_2	32 KB_4
256 KB_4	64 KB_8

L3 Configurations (size_associativity)	L3 Configurations (size_associativity)
1 MB_8	1 MB_16
2 MB_8	2 MB_16

Table 1. Few sets of configurations for L2 and L3 Cache.

3. **[CO1, CO2 &CO3, CO4]** Region-of-Interest (ROI) Statistics and Marker-Based Measurement: Experiments 1 and 2 collect statistics over the complete execution of `qsort_large`. In this experiment, measure only the sorting operation so that startup, file input, data preparation, and output do not affect the measured architectural statistics. For this experiment, use fixed configuration of caches is as given; L1 Data Cache (size = 16 kB, Associativity = 2, latencies = 2); L1 Instruction Cache (size = 16 kB, Associativity = 2, latencies = 2); L2 Cache (size = 512 kB, Associativity = 4, latencies = 10); L3 Cache (size = 1 MB, Associativity = 8, Latencies = 20).

Fixed ROI: the `qsort()` call that sorts the populated array; this is the ONLY region to be measured. It represents the main sorting computation and excludes input reading/data preparation before the `qsort()` call and result printing after the `qsort()` call.

Marker Placement: Modify `qsort_large.c` by placing the ROI markers exactly as follows:

1. Place the statistics-reset / ROI-begin marker immediately BEFORE the `qsort()` statement.
2. Keep the original `qsort()` statement unchanged.
3. Place the statistics-dump / ROI-end marker immediately AFTER the `qsort()` statement.

The required structure is therefore:

- `m5_dump_reset_stats(0, 0);`
- `qsort(array,count,sizeof(struct my3DVertexStruct),compare);`
- `m5_dump_reset_stats(0, 0);`

Do NOT place the markers inside `compare()`, inside the `qsort()` implementation/library code, around the input loop, around the `printf()` loop, or around the entire `main()` function. The markers must surround only the `qsort()` call shown above.

Deliverables

1. The assignment requires you to make observations on the L2 and L3 cache miss rate statistics report generated by each configuration for each CPU (RiscvTimingSimpleCPU() and RiscvO3CPU()) as you change the cache configurations.
2. Submit the following:
 - A. Configuration script for both systems given in question 1.
 - B. A separate graph (as shown in Figure 2) with the cache configurations you have considered and simulated on gem5 for both CPU models. (list all the configurations in a table). Compare the graphs which you have obtained after performing the simulations. Submit separate plots for L2 and L3 cache.
 - C. Report the observed simulation Ticks for each CPU model.
 - D. Explain your observations from the experiment performed in question 2.
 - E. Discuss the ROI findings from Experiment 3 and explain the differences between stats of full-run and ROI statistics.

Note: 2.B, 2.C, 2.D and 2.E should be uploaded in a single pdf. The Python/source script should be kept separately in the same folder. All these should be zipped with the folder name <your_name_roll_number_SA1> and uploaded in the classroom. Do not forget to “TURN IN” your assignment.

[1] <https://github.com/embecosm/mibench/tree/master/automotive/qsor>

[2] Hennessy, J.L. and Patterson, D.A., 2011. *Computer architecture: a quantitative approach*. Elsevier.

Steps to Install the RISC-V Toolchain to generate the Binary File:

Link for the toolchain: <https://github.com/riscv-collab/riscv-gnu-toolchain>

1. **Install the Dependencies:** You'll need some dependencies before you can build the RISC-V toolchain. Run the following commands:

```
sudo apt-get update
```

```
sudo apt-get install autoconf automake autotools-dev curl python3
```

```
python3-pip libmpc-dev libmpfr-dev libgmp-dev gawk build-essential bison
```

```
flex texinfo gperf libtool patchutils bc zlib1g-dev libexpat-dev
```

```
ninja-build git cmake libglib2.0-dev libsdl2-dev
```

2. **Clone the RISC-V GNU Toolchain Repository:** Clone the toolchain repository from GitHub.

```
git clone https://github.com/riscv/riscv-gnu-toolchain
cd riscv-gnu-toolchain
```

3. **Build the Toolchain:** You can build the for the RISC-V architecture using the following command.

```
./configure --prefix=/opt/riscv
make
```

This step will take some time to get completed. The binaries will be installed in [/opt/riscv/bin](#)

4. **Add the Toolchain to PATH:** Once the toolchain is built, you should add the directory to your PATH so that you can use the [riscv-unknown-elf-gcc](#) command from anywhere.

```
export PATH=/opt/riscv/bin:$PATH
```

You can also add this line to your ‘.bashrc’ or ‘.bash_profile’ to make the changes permanent.

```
echo 'export PATH=/opt/riscv/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

5. **Compile your Program:** Now, you should be able to compile your program.

```
riscv64-unknown-elf-gcc -o qsort_small.elf qsort_small.c
```

If you are using Docker, you can install the toolchain directly inside the container or use an existing Docker image with the RISC-V toolchain pre-installed.

Note: A demo and viva will be conducted for the assignment.